
Design layer

the whole layer, as one document

Contents

1 Correio	4
1.1 Foundation	4
1.2 System at a glance	5
1.3 Messages and delivery	6
1.3.1 Message model	6
1.3.2 Transport and evidence	7
1.3.3 Development mailbox and flow tests	9
1.4 Passwordless verification	10
1.5 Storage and runtime ownership	12
1.6 Application and Warden composition	14
1.7 Evidence and measurement	16
Glossary	16

ROOT DESIGN

Correio

1 Correio

1.1 Foundation

Correio supplies email delivery and independent passwordless verification for Gleam applications on Erlang.

GOAL

Compose transactional email without provider coupling

A caller constructs one validated message and selects a transport while retaining provider-specific capabilities and delivery evidence.

GOAL

Verify possession with one confirmed consumption

A caller authenticates an application-native subject through a short-lived magic link or email code whose consumption is atomic across concurrent callers.

GOAL

Make failure and scale measurable

Executable consumers establish transport, persistence, recovery, and concurrency behavior; repeatable benchmarks describe measured throughput and latency.

NO-GOAL

Own accounts or application sessions

Registration, account linking, business permissions, browser cookies, account discovery responses, and session publication remain application responsibilities.

NO-GOAL

Operate an inbound mailbox or identity provider

IMAP, POP, inbound mail processing, OAuth authorization-server behavior, WebAuthn, and authenticator TOTP are outside this package's contract.

INVARIANT

Possible transmission cannot become proven non-delivery

An uncertain transport outcome remains uncertain. Neither the adapter nor the library retries after possible transmission.

mechanism

INVARIANT

One command consumes a challenge

Only a confirmed atomic consumption yields verification evidence. Repeating the same recovery command can recover its receipt without authorizing a second consumption.

mechanism

INVARIANT

Expiry is checked at the serialization boundary

The store samples authoritative time after acquiring the challenge's serialization authority. A pre-lock clock sample cannot authorize an expired challenge.

mechanism

INVARIANT

Ordinary diagnostics exclude secrets

Descriptions and observations exclude provider credentials, challenge answers, message bodies, arbitrary application subjects, and raw foreign errors. Explicit delivery and VM inspection are outside that boundary.

mechanism

PRINCIPLE

Keep ownership visible

Constructors validate without starting processes. Runtime handles have explicit start and stop boundaries; injected pools and stores remain caller-owned.

PRINCIPLE

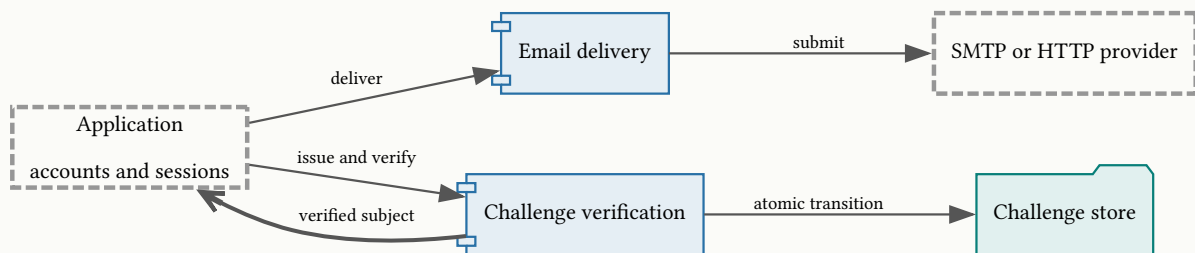
Preserve application-native values

Typed subjects and provider extensions compose through public contracts without importing implementation modules.

1.2 System at a glance

Message delivery and challenge verification have independent lifetimes. The application composes them.

VIEWPOINT CONTEXT-OWNERSHIP · ALTITUDE L1 · BOUNDARY Email and authentication composition



LEGEND

 **external system**
trust boundary

 **component**
owned runtime part

 **database**
persistent store

 **call**
solid directed link

 **dataflow**
weighted directed link

No email delivery is triggered implicitly by verification. The application decides how to compose a challenge into a message and how to publish an authenticated session.

Package composition

responsibility

Expose independent message, delivery, and challenge contracts with explicit effects.

interface

Public Gleam modules, a separate consumer, optional PostgreSQL and Postmark adapters, and local test peers.

interactions

SMTP uses a BEAM transport; HTTP uses caller-owned HTTP Gun; durable challenge storage uses caller-owned PostgreSQL resources.

invariants

No runtime dependency on Warden, Grind, Phoenix, Swoosh, or AshAuthentication. Oracles remain test-only.

failure

Construction, external refusal, uncertain effect, and unavailable infrastructure remain distinct outcomes.

- The message model is an immutable value model. The challenge is an identity-bearing aggregate with a bounded lifecycle. Their shared package does not create shared transaction authority.
- An OIDC identity and email verification evidence remain distinct inputs to an application's principal mapping. Correio never fabricates an issuer or OAuth token. [ADR-0001](#)
- A login journey issues a challenge, submits its message, confirms an intentional user action, consumes the challenge, and publishes an application session. A failure between these boundaries does not roll back an already accepted email or consumed challenge.

Package or module	Owned contract	Effect owner
correio/address, message, mime	Immutable construction, admission, and serialization	No network or database effects
correio/smtp and capture	Typed sender and delivery evidence	One SMTP exchange or explicitly started capture runtime
correio/passwordless	Challenge generation, bindings, and evidence	Injected semantic store
correio/passwordless/memory	Bounded local serialized rows	Explicitly started local runtime
adapters/correio_postgres	Durable atomic transitions and cleanup	Caller-owned named Pog pool
adapters/correio_postmark	Postmark request and response semantics	Caller-owned HTTP Gun client
correio/testing and adapters/correio_mailbox	Flow helpers and read-only development inspection	Caller-owned capture and HTTP server
apps/passwordless_demo	Local browser journey and principal mapping	Application state, HTTP listener, and sessions

1.3 Messages and delivery

A validated message describes content. A transport supplies observed submission evidence.

1.3.1 Message model

Immutable email

responsibility

Construct a [Message](#) and its [Envelope](#) without performing I/O.

interface

Validated address and header constructors; a small message constructor; composable text, HTML, recipient, reply-to, header, and attachment operations.

interactions

Serializers consume the same message through public projections. Applications own templates and localization.

invariants

At least one recipient and one body alternative exist at delivery admission. Sender and address fields reject line breaks and control characters. Reserved structural headers cannot be overridden through arbitrary headers. Blind recipients are present in the envelope and absent from MIME headers.

failure

Invalid addresses, headers, missing content, unsupported adapter capability, and size limits are typed refusals before network submission.

- Addresses preserve local-part case and normalize domain case. The supported address grammar is explicit and conservative; quoted local parts and SMTPUTF8 addresses fail explicitly unless an adapter and encoding contract support them. Display names support Unicode through MIME encoding.
- Plain text and HTML can coexist as multipart alternatives. Attachments carry bytes, content type, filename, and attachment or inline disposition; inline content has a content identifier. Envelope sender may differ from the visible From address.
- MIME uses CRLF, encoded Unicode headers, correctly escaped display names, stable multipart nesting, and transfer encoding suitable for arbitrary attachment bytes. Generated date, message identifier, and multipart boundaries are independently normalized for comparisons.

- Validation bounds recipient count, header length, body and attachment bytes, and total submission size before transport. Configuration can lower or explicitly raise documented bounds. Provider capabilities fail explicitly rather than silently dropping fields.

Value	Parts and refinement	Relationship
Address	ASCII dot-atom mailbox, normalized domain, optional Unicode display name	A message has one sender and one or more recipients
Message	Sender, recipients by role, subject, body, optional reply-to, headers, and attachments	One immutable message derives one envelope
Body	Text, HTML, or both nonempty alternatives	A message has one body value
Attachment	Filename, media type, bytes, and file or inline disposition	A message has zero or more attachments; inline disposition has one content identifier
Envelope	Sender and unique mailbox recipients, including blind recipients	Display names do not change recipient identity
Delivery evidence	Accepted receipt, proven no-send, definite rejection, or unknown outcome	One submission yields one observed outcome

- Defaults admit 100 recipients, 50 custom headers, 8 KiB per header value, 1 MiB of combined body content, 10 MiB per attachment, and 25 MiB of submission preparation. A conservative encoded-size estimate runs before serialization; exact encoded size is checked afterward. A message can therefore be refused even when its exact wire size would fit.
- Address literals and internationalized envelope addresses are refused. Attachment media types under message or multipart are refused because they require additional encoding semantics. Custom headers cannot override envelope, structural, authentication, or MIME headers. Custom values preserve ASCII syntax and reject physical lines longer than 998 bytes. Non-ASCII custom values are refused; subjects and display names retain Unicode support.

BEHAVIOR

Blind recipients stay private

Given A message includes blind recipients.

When The application prepares it for delivery.

Then The delivery envelope includes those recipients.

Then The serialized message excludes the blind-recipient header.

level: boundary area: Message construction

BEHAVIOR

Invalid content is refused before submission

Given A message violates a structural or configured size requirement.

When The application requests delivery.

Then Delivery returns a typed refusal.

Then No provider request occurs.

level: boundary area: Message admission

1.3.2 Transport and evidence

Delivery operation

responsibility

Submit one admitted message and retain **Delivery evidence**.

interface

A typed sender accepting one message; SMTP, HTTP-provider, and capture implementations capture validated configuration and provider-specific options.

interactions

The caller owns HTTP pools and scheduling. SMTP owns only its admitted exchange. Capture state has explicit ownership and bounded capacity.

invariants

A provider acceptance receipt is not inbox delivery. Possible submission is not retried internally. Per-operation deadlines and input bounds limit retained work.

failure

Construction refusal, proven no-send, definite provider rejection, accepted submission, and unknown outcome are distinguishable. Unsupported features fail before submission.

- SMTP requires certificate-chain and hostname verification with STARTTLS-required or implicit TLS configuration. Plaintext is an explicitly named local-test option. Authentication cannot silently downgrade to unauthenticated delivery. Response codes govern acceptance and rejection.
- SMTP envelope recipients include To, Cc, and Bcc. Any recipient refusal aborts before DATA, so the adapter does not partially submit message content. A lost reply after DATA remains unknown.
- HTTP provider calls use HTTPS, bounded response bodies, explicit destination policy, and caller-owned HTTP Gun lifetime. A provider-specific idempotency key is meaningful only under that provider's documented semantics. Generic message identifiers do not establish deduplication.
- Postmark uses its fixed email endpoint and typed options for message stream, tag, metadata, and tracking. An unsupported envelope sender or provider size limit fails before submission. A successful HTTP status alone is insufficient; the response must confirm provider success and identify the message. Ambiguous server failures remain unknown.
- Delivery scheduling, durable outbox storage, bounce webhooks, and reconciliation are application integrations. Retrying an unknown outcome requires an explicit duplicate-delivery policy or destination-supported idempotency.
- A capture adapter records complete normalized messages without network I/O. Its test API supports explicit clearing and bounded retrieval without global cross-test state.

BEHAVIOR

Lost submission acknowledgement stays uncertain

Given A provider may have accepted a submitted message.

When Its acknowledgement is unavailable.

Then Delivery reports an unknown outcome.

Then The library makes no replacement submission.

level: boundary area: Delivery recovery

BEHAVIOR

Untrusted transport is refused

Given A configured secure transport cannot authenticate its peer.

When The application requests delivery.

Then The exchange fails without transmitting message content or credentials over an unauthenticated connection.

level: boundary area: Transport security

1.3.3 Development mailbox and flow tests

Explicit development inspection

responsibility

Inspect locally captured mail while testing an application flow.

interface

Capture entries expose increasing identifiers and complete messages. An opaque checkpoint binds one capture and its current position. Testing helpers select matching messages after a checkpoint with a bounded wait. The optional `correio_mailbox` package handles Gleam HTTP requests at an application-selected mount path.

interactions

The application injects the capture sender into its delivery path and mounts the handler in its development server. The application owns server lifetime and access policy. Core capture has no HTTP dependency. [ADR-0002](#)

invariants

Inspection never sends mail or consumes a challenge. Clearing messages does not rewind identifiers or checkpoints. Helpers cannot match mail before their checkpoint. The read-only UI never activates message links automatically.

failure

Capture unavailability, capture deadline, invalid wait duration, and no matching message before timeout remain distinct typed results. Missing UI entries return not found; unavailable capture returns service unavailable.

- Entry identifiers are unique within one capture lifetime. Checkpoints retain their capture binding. Entries remain bounded by capacity and disappear on clear or termination.
- Testing predicates run in the caller process. Helpers return the earliest retained match after their checkpoint. Zero wait performs one inspection under the capture query bound of one second without polling. Wait durations range from zero through sixty seconds. Positive waits use monotonic deadlines; application predicate execution time remains caller-owned. Recipient matching uses normalized envelope mailboxes, including blind recipients.
- The mailbox lists a bounded page of recent entries and resolves details by stable identifier. It displays metadata, plain text, escaped original HTML source, and sandboxed HTML preview. The preview preserves only an explicit set of formatting tags and removes all attributes; active tags, links, images, and styles have no preview authority. Preview content cannot run scripts, submit forms, navigate the parent, or fetch remote resources. Copying and opening a sign-in URL remains an intentional user action.
- Responses prohibit caching and external framing. No capture is global and no listener starts implicitly. Captured messages contain development secrets; the application mounts the handler only in an explicitly enabled development environment.

BEHAVIOR

A flow sees only its new messages

Given A capture contains earlier messages and a test records a checkpoint.

When The flow sends a matching message and the helper inspects after that checkpoint.

Then The helper returns the new matching message.

Then Earlier messages cannot satisfy the match, including after a clear.

level: boundary area: Development testing

BEHAVIOR

Waiting terminates explicitly

Given A test waits for a matching message after a checkpoint.

When The deadline elapses or the capture stops.

Then The helper returns a typed timeout or capture failure without including message content in the error.

level: boundary area: Development testing

BEHAVIOR

Mailbox inspection has no authentication effect

Given A captured message contains a sign-in secret and untrusted HTML.

When A developer opens its mailbox detail page.

Then The developer can inspect the message.

Then No sign-in URL is requested automatically and no message script, form, or remote resource executes.

level: interface area: Development mailbox

1.4 Passwordless verification

A challenge proves possession for one explicit purpose. It does not decide account or session policy.

Challenge service

responsibility

Issue and verify a **Challenge** for a caller-owned subject and configured scope.

interface

A validated service over a typed challenge store; issue, verify, revoke, and recovery operations; opaque challenge and **Verification evidence** values with explicit projections.

interactions

The application chooses subjects and destinations and delivers secrets. Store adapters serialize challenge transitions and sample time. Cryptographic randomness and keyed verification use OTP primitives.

invariants

A challenge binds subject, scope, purpose, destination, secret verifier, expiry, and attempt budget. A verification result exists only after confirmed consumption. No user enumeration or account linking occurs inside the package.

failure

Malformed, missing, expired, revoked, exhausted, and already consumed challenges are refused. Unavailable storage and uncertain mutation are distinct infrastructure outcomes.

- Magic-link secrets contain at least 256 bits of cryptographic randomness and use URL-safe encoding. Email codes use unbiased cryptographic generation and a configurable bounded attempt budget. Code verifiers are keyed hashes bound to the challenge and service scope; a database reader cannot enumerate codes without the application key. Raw answers are never persisted in challenge rows.
- Defaults use a ten-minute validity interval, five failed attempts, and twenty-four hours of terminal receipt retention beyond validity. Configuration is validated before any effect. A challenge identifier is independent of its answer and is not authentication authority.
- The application can use a native subject type. A durable adapter requires a caller-selected codec for that type and refuses incompatible stored values explicitly. Subject serialization is not message serialization.
- Each issue command has a fresh identity; uncertain insertion can recover the same challenge without generating another answer. Each verification command has a fresh identity; recovery reads or repeats only that command's atomic transition and preserves its original authentication time.
- Independent successful requests cannot reuse a consumed challenge. A duplicate recovery command may return the same receipt while retention permits it. Recovery never extends the authentication lifetime or creates another consumption.
- Wrong answers atomically spend an attempt only on an active challenge. Exhaustion withdraws further verification. Missing or malformed identifiers cannot create rows. A correct answer arriving after expiry is refused even if it waited for a lock while unexpired.
- Revoke serializes with consumption. A revoke that wins prevents consumption; a consumption that wins remains historical evidence, and application session invalidation remains a separate responsibility.
- Resending an existing message does not extend challenge expiry. Issuing another challenge does not silently revoke older challenges; applications explicitly revoke when their resend policy requires it. Secret delivery delayed beyond expiry cannot authenticate.
- Opaque values prevent accidental construction through the public Glean API. Ordinary error descriptions redact secrets; runtime inspection and explicitly requested secret projection are outside that guarantee.

Challenge status

active consumed revoked exhausted expired

Challenge

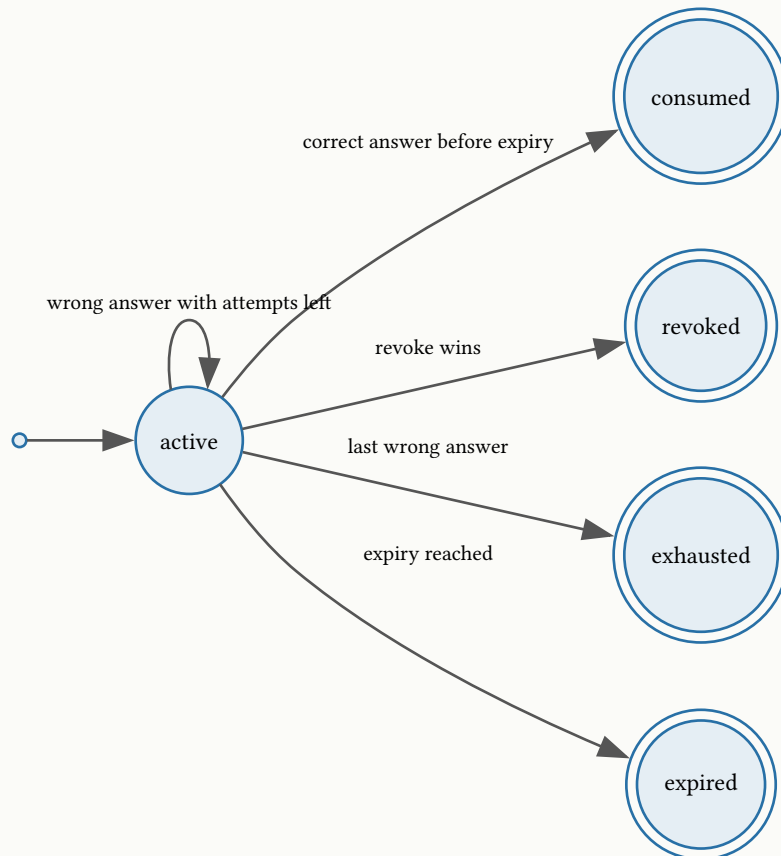
aggregate stateful Authentication owned by Passwordless verification

One opportunity to prove possession.

ATTRIBUTES

- derived** **Identity and binding** · Challenge binding
Random identifier plus the application subject, destination, scope, and purpose captured at issue.
- authored** **Verification policy** · Validity and attempt limits
Validated service policy captured for this challenge.
- derived** **Status** · Challenge status [challenge-status](#) · [challenge-lifecycle](#)
The result of serialized verification, expiry, or revocation.
- observed** **Consumption receipt** · Optional command and authentication time
The confirmed winning command and the time at which its authentication evidence arose.

STATE MACHINE Challenge consumption



No transition recreates an active challenge. Recovery observes the original command rather than resetting this lifecycle.

- The public lifecycle describes authentication authority. Storage represents exhaustion with zero remaining attempts and records observed expiry as a terminal status. Retention expiration can discard a consumption receipt without recreating an authentication opportunity.
- The challenge service captures one store, scope, purpose, signing key, and validated policy. An issue command captures one native subject, destination, independent random challenge identity, keyed answer verifier, absolute expiry, retention deadline, and attempt budget.

- A verification command carries a fresh command identity and the original challenge binding. Failed command identities are retained within the attempt bound, so recovery of a wrong answer does not spend another attempt. Confirmed consumption stores the winning command identity and original authentication time.
- A verification receipt references one challenge and one winning command. The opaque public evidence projects only the subject, destination, scope, purpose, challenge identity, and authentication time. Recovery handles carry the original operation and are sensitive application state; generic runtime printing is outside redacted diagnostics.
- Validity is 1 millisecond through 24 hours, attempts are 1 through 100, email codes contain 6 through 10 digits, and retention is 1 millisecond through 30 days. Keys contain 32 through 4096 complete bytes. Scope, purpose, and destination are nonempty, at most 320 bytes, and exclude null and line-break characters.

BEHAVIOR

Concurrent verification has one winner

Given An active challenge has a correct answer.

Given Storage remains available, no revocation competes, and the first serialized verification occurs before expiry.

When Independent callers verify it concurrently.

Then Exactly one consumption succeeds.

Then Every other caller is refused.

level: boundary area: Consumption

BEHAVIOR

Expiry is checked after waiting

Given A verification waits for serialization while its challenge expires.

When The store obtains authority to consume it.

Then The challenge is refused as expired.

level: boundary area: Consumption expiry

BEHAVIOR

Wrong answers spend a bounded budget

Given An active challenge has a finite attempt budget.

When Callers submit wrong answers concurrently.

Then The store counts the failed attempts atomically.

Then Exhaustion prevents subsequent authentication.

level: boundary area: Attempt admission

BEHAVIOR

Uncertain acknowledgement cannot mint evidence

Given A challenge mutation may have committed without acknowledgement.

When The caller receives the uncertain result.

Then The result carries recovery authority for the original command.

Then Verification evidence is withheld until that command's consumption is confirmed.

level: boundary area: Storage recovery

1.5 Storage and runtime ownership

Adapters enforce atomic semantic operations, including time and attempt accounting, rather than exposing a generic compare-and-set as sufficient evidence.

Challenge store

responsibility

Serialize issue, verify, recovery, and revoke while retaining terminal receipts.

interface

A generic typed store port with immutable operation inputs, closed results, explicit unavailable and unknown outcomes, and bounded cleanup.

interactions

Memory storage owns explicit bounded process state. PostgreSQL storage captures a caller-owned connection pool and a subject codec.

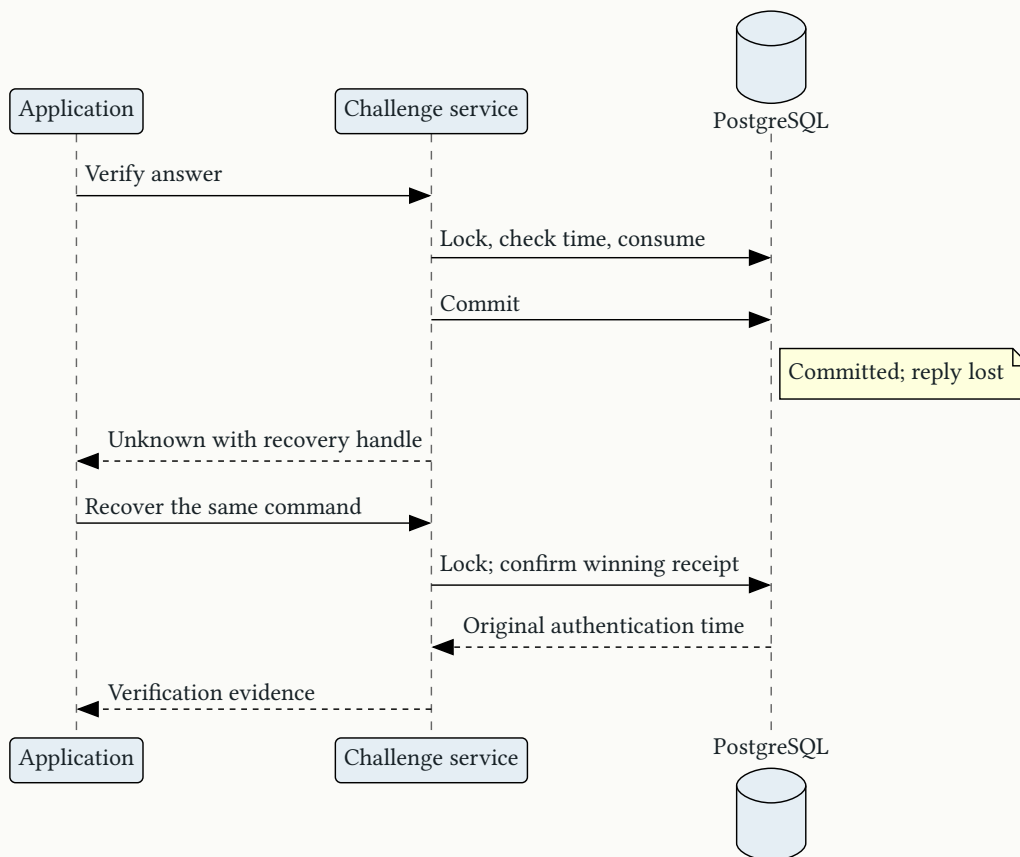
invariants

Expiry is sampled under serialization. Different challenges can progress independently. Recovery identities and secret verifiers are bound to the original challenge and scope. Cleanup cannot remove still-live authentication opportunities or receipts inside their configured retention.

failure

Capacity, decoding, availability, contention deadline, and uncertain commit are explicit outcomes. A timeout never proves rollback.

- The memory adapter is for local use and tests; restart loses its data. It has explicit capacity and stop semantics. Its clock can be injected for deterministic tests, but is sampled inside the serialized operation.
- The PostgreSQL adapter uses row-level serialization and a fresh database clock sample after lock acquisition. A transaction either records one complete transition or records none. Separate pooled connections establish multi-process concurrency; one connection hidden behind a shared test transaction does not prove this property.
- Issuance reads the store's authoritative clock once to bind immutable absolute expiry and retention into its command. Insertion refuses an already expired command. Recovery cannot renew those timestamps or reinsert an expired opportunity after cleanup.
- Database schema constraints protect challenge identity, attempt bounds, legal status, and receipt consistency. Queries use parameterized values and fixed table identifiers.
- Terminal rows remain until validity plus configured retention. Recovery checks the retention deadline even if cleanup has not run. Cleanup is explicit, bounded, and indexed; it never runs as an unowned background process. Persisted timestamps use a documented precision and unit.
- Caller-owned pools outlive admitted operations. Store calls and network exchanges have finite deadlines. Stopping a local runtime reports its actual completion boundary and never claims rollback of a timed-out external operation.
- Horizontal scale comes from independent application nodes sharing the authoritative PostgreSQL rows. Benchmarks separately measure uncontended different-challenge traffic and same-challenge contention.
- The PostgreSQL constructor accepts a named Pog pool, not an arbitrary connection. This type boundary prevents the adapter from committing an enclosing application transaction. Initial migration creates the fixed `ecarta_challenges` table and retention index explicitly. The table identifiers and verifier domain `ecarta-passwordless-v1` remain stable across package renames. [ADR-0003](#) Timestamp columns use integer Unix milliseconds.
- The memory store bounds retained rows to its configured capacity, at most one million. Cleanup accepts a deletion limit from 1 through 1000. PostgreSQL cleanup orders the retention index and skips locked rows. Memory cleanup scans its bounded retained state.
- The authoritative clock must be trustworthy. Observing expiry persists a terminal refusal so later clock rollback cannot revive that observed opportunity. A clock that moves backward before expiry is observed remains an operational clock-discipline risk.
- The application protects its signing key, database write authority, and trusted store implementation. The public store port is an adapter-author contract; a malicious injected store can forge receipts. Rotating to an unrelated signing key invalidates outstanding answers, so deployment retains the original key through the intended challenge lifetime or explicitly accepts revocation.



Recovery confirms one consumption. Application session publication remains a separate effect.

1.6 Application and Warden composition

The retained consumer exercises ordinary and advanced use without transferring application authority.

Separate consumer

responsibility

Demonstrate email login with native account values, explicit sessions, and truthful recovery.

interface

HTTP issue, landing, confirmation, and session routes; typed mail and challenge configuration; a compiled Warden identity projection recipe.

interactions

Local SMTP and HTTP peers capture real exchanges; disposable PostgreSQL persists challenges; the application owns account lookup and session storage.

invariants

Landing GET does not consume a challenge. Confirmation requires POST and application origin protection. Unknown accounts receive the same public issue response. Correio evidence and Warden OIDC identity remain distinct authentication methods.

failure

Provider refusal, expiry, wrong answer, storage outage, and failed session publication have explicit application responses. No error page exposes an answer or credential.

- The consumer uses a trusted configured origin when composing links. It does not derive callback destinations from untrusted Host or return-to values. It removes secrets from the address bar before rendering third-party resources and uses a restrictive referrer policy.
- The default consumer flow binds confirmation to the initiating browser. A separately exercised code flow permits intentional cross-device entry under its own explicit policy. The library does not infer browser policy from an email address.

- Per-destination and per-origin admission limits belong to the application or shared edge authority. The library's per-challenge attempt limit is necessary but does not substitute for those limits.
- Session publication after consumption uses the application's own idempotent publication identity. Retrying that publication uses retained verification evidence, never a fresh verification of the consumed answer.
- The initiating-browser cookie uses SameSite=Lax so a top-level navigation from webmail carries the binding. Session cookies use SameSite=Strict. Both cookies are HttpOnly. The loopback HTTP fixture omits Secure; a deployed HTTPS application sets it.
- Cross-device code entry creates a new awaiting-answer state. It cannot inherit confirmed evidence or a recovery handle from another browser. The receiving browser must prove the code through its own command.
- An uncertain verification acknowledgement retains its recovery handle in the initiating browser's pending state. Another intentional confirmation recovers that same command before publishing a session. Infrastructure failure does not become a false credential refusal.
- The retained browser application is a bounded local demonstration. Its single-exchange HTTP listener and ephemeral session state do not establish scalable production HTTP serving. Shared PostgreSQL challenge consumption supplies the package's horizontal authentication authority.

BEHAVIOR

Link inspection does not authenticate

Given A valid login link has been delivered.

When A browser or mail scanner opens it.

Then The application presents an intentional confirmation action.

Then The challenge remains unconsumed until confirmation.

level: interface area: Browser confirmation

1.7 Evidence and measurement

Each check names its authority and its limits.

Evidence	Contract exercised	Limit
Gleam compiler and separate consumer	Ordinary and advanced public calls with application-native types	Compilation alone proves no network or database behavior
Swoosh oracle	Normalized recipients, bodies, headers, attachment bytes, and MIME structure	No inbox-delivery or provider-wide parity claim
Phoenix oracle	Sequential challenge acceptance, expiry, purpose and destination binding, and reuse refusal	Generated application policy is compared only where Correio's contract matches
AshAuthentication oracle	Concurrent single-use consumption on separate PostgreSQL connections	Upstream success does not establish Correio's store correctness
Native transport and database faults	TLS refusal, ambiguous submission, commit uncertainty, lock expiry, restart, and concurrency	Scripted local peers do not qualify every remote provider
Benchmarks	Message preparation, verification, database throughput and latency, and contention	Recorded host, concurrency, workload, and revision delimit the result

- An **Oracle** manifest pins version, exact source revision, license, fixture inputs, command, normalization, and intentional differences. Oracle runners execute the pinned implementation; an imitation of its algorithm is not independent evidence.
- Normalized MIME comparisons ignore generated date, message identifier, boundary strings, and insignificant folding while preserving recipient roles, media types, body alternatives, disposition, content identifiers, and decoded bytes.
- Oracle suites compare deterministic scenarios. Live SMTP providers and existing user databases are unnecessary. Local peer behavior, real protocol exchange, and database fault schedules remain separate receipts.
- Benchmarks report warmup, repetitions, sample count, throughput, latency percentiles, and errors. Same-challenge contention reports accepted and refused counts separately; duplicate refusals are not counted as successful authentications.
- The complete gate includes package and consumer compilation, tests, format validation, oracle comparisons, native end-to-end cases, and design integrity. Benchmarks are reproducible measurement commands rather than machine-independent pass/fail throughput thresholds.

Glossary

Every term the layer declares, with the context that owns it.

Challenge

owned by root

A bounded opportunity to prove possession of a secret delivered to a particular address for one subject, scope, and purpose.

Consumption

owned by root

The atomic transition that grants one verification command the challenge's authentication evidence after checking its secret, attempt budget, and expiry.

Delivery evidence

owned by root

The observed distinction between provider acceptance, definite refusal or non-transmission, and possible transmission without a confirmed outcome. It does not establish inbox delivery.

Envelope

owned by root

The sender and complete recipient set submitted to the transport, including blind recipients that are absent from rendered headers.

Message

owned by root

A validated immutable description of email recipients, content, and attachments, independent of a delivery provider.

Oracle

owned by root

A pinned external implementation used to judge an explicitly named subset of observable behavior.

Recovery

owned by root

An opaque retained command that resolves an uncertain storage acknowledgement without issuing another challenge or granting another consumption.

Retention

owned by root

The interval during which terminal challenge records preserve replay refusal and recovery receipts after their authentication opportunity ends.

Verification evidence

owned by root

Opaque evidence of confirmed challenge consumption carrying the original application subject, destination, purpose, scope, and authentication time. It is not an OIDC identity or a permission grant.